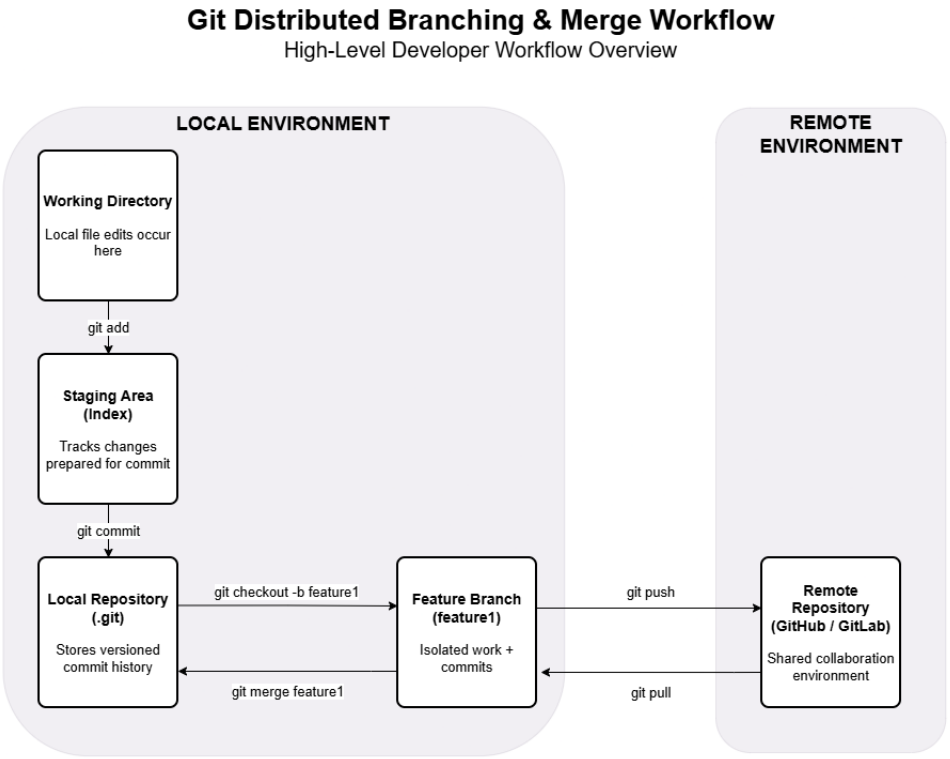
Git Distributed Branching & Merge Workflow

High-Level Developer Workflow Overview

Overview

This document illustrates a high-level distributed Git workflow commonly used in collaborative software development environments. The model demonstrates how changes move through the Working Directory, Staging Area, and Local Repository, how feature branches enable isolated development, and how synchronization occurs between local and remote repositories (e.g., GitHub or GitLab).



Local Workflow

Development begins in the **Working Directory**, where files are created or modified. Changes are selectively prepared for version control using:

git add

This command moves updates into the **Staging Area (Index)**, which acts as a preparation layer for commits.

Once changes are staged, the following command records a versioned snapshot:

git commit

The commit is stored in the **Local Repository (.git)**, preserving a structured project history and enabling traceability of changes.

Branching & Merging

To support isolated development, a feature branch can be created using:

git checkout -b feature1

The **Feature Branch** allows developers to make independent commits without affecting the primary branch. This separation enables parallel workstreams and reduces risk to the stable codebase.

When development is complete and reviewed, the branch is integrated back into the main line of development using:

git merge feature1

This operation consolidates the feature branch changes into the Local Repository's primary branch.

Remote Synchronization

Git operates as a distributed version control system. While the Local Repository stores complete project history, collaboration occurs through a shared **Remote Repository** (e.g., GitHub or GitLab).

To upload local commits to the shared repository:

git push

To retrieve updates made by collaborators:

git pull

This distributed architecture enables teams to work concurrently while maintaining a shared source of truth and structured version history.

Summary

This workflow demonstrates the core principles of distributed version control:

- Separation of working, staging, and commit layers
- Isolated feature development through branching
- Controlled integration through merging
- Bidirectional synchronization with a remote repository

Together, these mechanisms support traceability, collaboration, and controlled software delivery in modern engineering environments.